BSc (Hons) in Information Technology Specialized in AI

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1

Practical 05

Name: Induma N.G.A
IT num: IT24104249

Theoretical Evaluation Answers

1. Reachability vs. Feasibility (Apply)
In Step 3.2, you modified the A* algorithm's neighbor-generation loop. How does your new code differentiate between checking if a node is "Reachable" (like a wall collision) versus checking if it is "Feasible"? Explain how the Knowledge Base enforces feasibility.

- **Reachability** is a physical grid property checking whether a coordinate is unblocked by hard geometric obstacles (such as boundary walls or static barriers).
- **Feasibility** is a logical safety property determining whether an open, reachable coordinate is safe to occupy under current threat conditions.
- **Knowledge Base Enforcement**: The Knowledge Base enforces feasibility by processing active environmental percepts through forward_chain(). If logical rules deduce a hazard condition (such as 'Retreat'), the node is flagged as infeasible and discarded from A^* expansion, even though it remains physically reachable.

2. Declarative vs. Procedural Paradigms (Analyze)
Why did we build a KnowledgeBase class with a forward_chain() loop instead of simply

writing if 'TargetVisible' in percepts and 'HasDust' in percepts: directly inside the agent's movement code? What happens to the agent's code complexity if we add 50 new game rules?Declarative Knowledge Base: Separates *what the system knows* (facts and rules) from *how it reasons* (the inference engine).

- **Procedural Code**: Embeds conditional rules directly into program control flow via nested if-else branches.
- **Code Complexity with 50 New Rules**: In a procedural system, adding 50 rules creates an unmanageable tree of nested if-else logic, drastically increasing bug risk and code complexity. In a declarative Knowledge Base, adding 50 rules requires simply adding 50 calls to tell_rule(), leaving the core forward_chain() loop and pathfinding algorithm entirely unmodified.

3. Modus Ponens & Horn Clauses

- **Horn Clauses**: A Horn Clause is a logical rule containing a conjunction of premises leading to a single positive conclusion: $(P_1 \wedge P_2 \wedge \ldots \wedge P_n) \rightarrow Q$.
- **Modus Ponens**: A foundational inference rule stating that if every premise P_i in an implication is known to be true, then conclusion Q must also be true.
- **Implementation Mapping**: The forward_chain() while loop mathematically executes Modus Ponens by iterating over all stored Horn Clauses, evaluating if all(premise in self.facts for premise in premises), and appending conclusion to self.facts whenever all premises hold true. It continues this cycle until no further Modus Ponens deductions can be made.